Nitsche speedups 1, 3, 4, 5 and 6: changes and measured gains

What each change does to the code, how it was checked against `53ba400`, and what it gained on TPV205, TPV104 and TPV102 at p = 1 and p = 2 · revision 1.1, 18 Oct 2026

Revision 1.1, 18 Oct 2026. Adds the Vista measurement (§7): TPV205 200 m, p = 2, on the gold's 2304 ranks, and the bench checks on 16 Vista ranks. The changes are now on GitHub (`5cb2500`, `2673c50`; Vista jobs `6d41ba7`; bench fix `0c83fb5`), and the code links point at `0c83fb5`. Old §7 and §8 are now §8 and §9; §1, §6, §8 and §9 are updated. **Revision 1** (17 Oct) was the first version. Apart from §7, every timing comes from the cloud VM (4 cores, one run per core), with the old binary built from `53ba400` and the new one from the reviewed tree. Item numbers are those of the profiling report (`nitsche-profiling-plan-rev1.pdf`, cited as "profiling §n").

1. Summary

On Vista, at the gold's 2304 ranks, the new calls make a TPV205 200 m p = 2 time step 1.65 times faster (27.0 ms to 16.4 ms on the slowest rank) and leave the answers unchanged. With the flags off the 16 stations match gold 1075916 bit for bit; with them on they differ from that run by at most $6.6 \cdot 10^{-11}$. A fault frame falls from 0.41 s to 0.13 s. The 1 s test took 321 s before and 244 s after (1.32 times, because setup does not speed up); scaled to the gold's 12 s it would take about 1640 s instead of the gold's 2670 s (§7).

All five items are implemented, and the results agree with `53ba400` to round-off. On the cloud VM with 4 ranks, a TPV205 run takes 39.0 s before and 27.5 s after at p = 1 (1.42 times faster), and 51.0 s before and 43.7 s after at p = 2 (1.17 times). TPV104 and TPV102 gain 1.10 to 1.82 times (§5). The gains come from three places.

1. **Fault output (item 1).** One fault frame costs 20 times less at p = 1 (0.129 s to 0.0063 s per frame on the 2000 m mesh). At the size of a 200 m fault (104 882 triangles) the bench gives 1.15 s per frame for the old writer and 0.454 s for the new one on one rank (2.5 times). Most of what is left at that size is deflate: with no filter the new writer takes 0.023 s, but no deck setting turns the filter off yet (§5). The values in the file are bitwise identical to the old writer's.
2. **Time per step (items 3 and 4).** Without output, a step takes 20% less time at p = 1 and 15% less at p = 2. Switched off one at a time, the per-face fault operators (item 3) account for 17% and 13% of the old step time, and the fused stiffness products (item 4) for 11% and 9%.
3. **Memory (item 5).** Memory while stepping falls from 3.03 to 1.18 GiB at p = 1 and from 8.90 to 2.84 GiB at p = 2 (4 ranks together). The serial forms of K and K_η are freed after assembly, and the two hypre matrices once the fused copy exists. The setup peak falls by 14 to 19% at p = 1. At p = 2 it is set by the moment the fused copy is built next to both hypre matrices, and it ends up close to the old peak: 7% lower for TPV205 and 3.5% higher for TPV104 and TPV102 on their 3000 m mesh (§3.2). Setup time does not change.

The fault-weighted partition (item 6) is in and off by default. On the cloud meshes the fault work is under 5% of a rank's step, so it changes nothing measurable there. For the 200 m

meshes, at the rank counts of the Vista golds, the model predicts a step-time gain of 1.03 to 1.06 times from the weighting (§6). The default stays at 0, MFEM's own cut: on Vista the weighted cut gained under 1% per step and used more memory on the first node (§7).

Item	What changed	Measured gain, TPV205, 4 ranks	Results
1	Fault frames: one gather of the changing fields, values written straight to the file, frame-sized HDF5 chunks, constant fields stored once	20× per frame at p = 1, 30× at p = 2; wall 6.2 s less at p = 1	File values bitwise identical
5	No serial form, BoomerAMG or CG for the explicit step; hypre matrices freed once fused	Stepping memory 2.6× and 3.1× lower; peak 14 to 19% lower at p = 1, −7 to +3.5% at p = 2	Unchanged
3	Fault traction and slip load from dense per-face operators built once	17% (p = 1) and 13% (p = 2) off a step; the calls themselves 23 to 48× and 47 to 79× faster	Round-off
4	$Ku + K_\eta v$ in one sweep over the union pattern, one halo message	11% (p = 1) and 9% (p = 2) off a step; the products 1.09 to 1.22× faster	Round-off
6	METIS weight $1 + w$ per fault-face side (`--fault-weight w`), cost model printed by the driver	None measurable on 4 ranks; 1.03 to 1.06× predicted at 200 m	Round-off

On the cloud VM a TPV205 p = 2 step without output takes 15% less time with the new calls on 4 ranks (2000 m mesh); on 2304 Vista ranks (200 m mesh) it takes 39% less (§7). Vista did not run the items one at a time, so their separate shares there are not measured.

2. Where the changes sit

Figure 1 follows one run of `seas_nitsche_dyn_driver` and marks each place a change enters. Setup changes the partition (item 6), what the assembly keeps (item 5), and builds two new structures (items 3 and 4). Each step then calls the new operators, and each fault frame goes through the new writer (item 1). Everything else in the step is unchanged: the friction closure, the state update, the mass solve and the corrector.

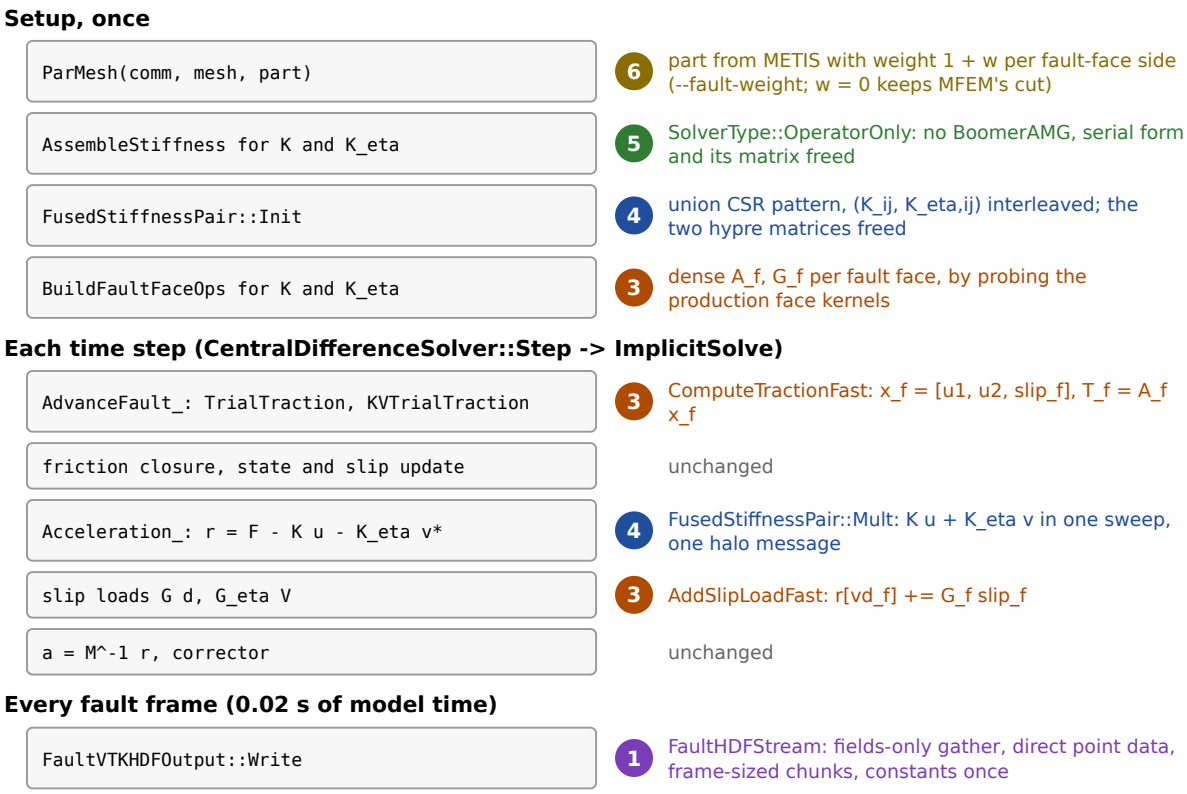


Figure 1. The run, with the changed calls marked by item. Grey rows are unchanged.

Table 2 lists every function that was added or changed. Each one is described in §3 with its inputs, outputs and pseudocode.

Item	Function	Change
1	VTKHDF::SetChunkToFirstAppend, VTKHDF::SavePointData (MFEM fork)	New, additive: chunk a new dataset to its first append; write point data from a plain array
1	vtkhdf::FaultHDFStream (`Write`, `Open_`, `Save_`)	New fault-frame writer; the old one stays for the other drivers
1	BuildFaultTriangleMesh	Factored out of `FaultHDFState::Init`, unchanged
1	FaultVTKHDFOutput (driver)	Uses the new writer; `--fast-fault-output 0` keeps the old one
5	SolverType::OperatorOnly, elasticity_operator_assembly.inl:315, ReleaseStiffness, ParStiffness	New solver type with no solver; frees the serial form; frees and re-assembles on demand
3	BuildFaultFaceOps, BuildFaultNbrExchange_, FaultNbrExchange_	New: build A_f, G_f per fault face, and the exchange of the elements next to shared fault faces
3	ComputeTractionFast, AddSlipLoadFast	New per-step calls
3, 4	TrialTraction, SlipLoad, KVTrialTraction, KVSlipLoad, Acceleration_, UseFastFault_, UseFused_	Call the new operators; fall back to the old calls when switched off
4	FusedStiffnessPair (`Init`, `Mult`, `SubsetMap_`, `Union_`)	New: union-pattern copy of K and K_η, fused product
6	driver partition (driver)	Weighted METIS cut when `--fault-weight` is above 0
all	driver flags, cost model (driver); bench_nitsche_kernels.cpp	Flags, step and fault timers, cost model; bench sections [F2], [T2], [P], [H]

The driver flags are `--fast-fault`, `--fused-spmv` and `--fast-fault-output` (each 1 by default, 0 restores the old call), and `--fault-weight w` (0 by default). The flags exist so that each item can be measured alone and so that a doubt about any one of them can be settled by switching it off; the old code paths are untouched.

3. The changes, item by item

3.1 Item 1: fault output

Why it was slow. The fault frame is written by rank 0 alone, and it took 11 to 26% of the Vista gold runs' wall time, 0.48 s per frame at 200 m (profiling §4.1). Most of a frame is spent inside HDF5, not in the gather. The profiling report found the per-cell field evaluation to be about 60% of a large frame and deflate about 25% (profiling §4.5). The second cost has a cause that is easy to remove. The data collection writes each field into a dataset chunked in 0.5 MB blocks with deflate on. A frame appends far less than 0.5 MB, so every append reads the last partly filled chunk back, inflates it, adds the new rows and deflates it again, for each of the 12 fields and for the step-offset datasets. The bench's [H] section shows the effect at the 2000 m size: 125 ms per frame with deflate 1, 87 ms with deflate 0 (which still recompresses, at level 0) and 15 ms with no filter.

What changed. The new writer, vtkhdf::FaultHDFStream, writes the same `fault.vtkhdf` with less work per frame (Figure 2). It does four things differently.

1. The geometry crosses to rank 0 only at the first frame. Later frames send the changing fields in one `MPI_Gatherv`, field by field within each rank's block.
2. Rank 0 writes the values with the new VTKHDF::SavePointData: each cell value goes to the cell's 3 points, which is exactly what `SaveGridFunction` writes for an L2 p = 0 field. The 12 `GridFunction` copies and the per-cell `GetVectorValues` calls are gone.
3. New datasets are chunked to their first append (VTKHDF::SetChunkToFirstAppend). A chunk then holds one frame, so an append writes whole new chunks and never reopens an old one. The chunk is at least 1024 rows and at most the old 0.5 MB.
4. `param_a` and `param_Dc` never change, so they are written at the first frame only. Their `Steps/PointDataOffsets` entries stay at 0 for every later step, and VTKHDF readers (ParaView, `tpv102/check_fault_vtkhdf.py`) then read the first frame's values at every step.

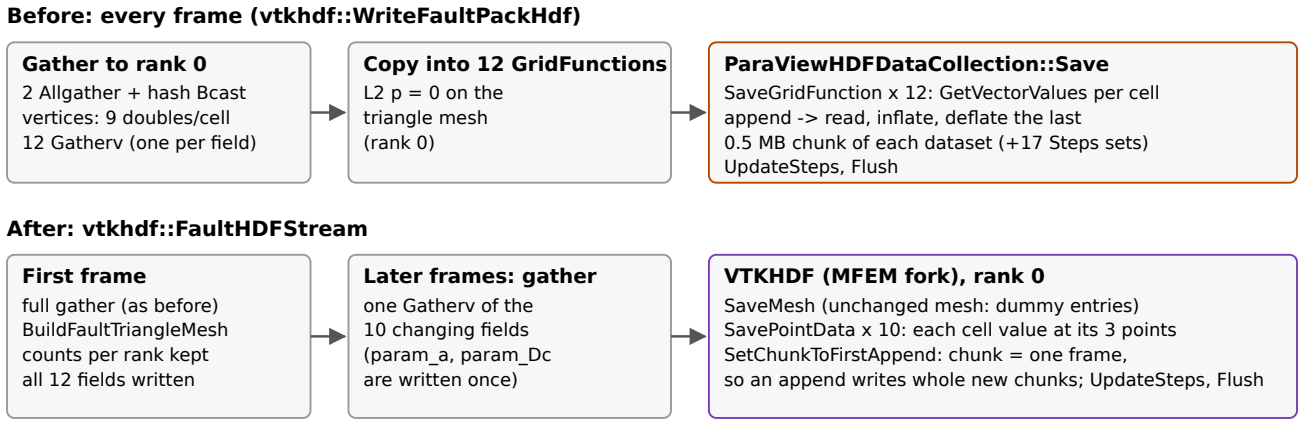


Figure 2. One fault frame before (the SAFS writer, still used by the other drivers) and after.

Functions.

- `FaultHDFStream::Write(prefix, time, local, constant, rank, nranks, comm)` (fault_vtkhdf_writer.hpp:346). Collective. Input: this rank's fault pack (triangles, vertices, 12 per-cell field arrays) and a flag per field saying whether it is constant. Output: one more step in `<prefix>/fault.vtkhdf` on rank 0. It checks that the cell count and field list never change.
- `FaultHDFStream::Open_` (fault_vtkhdf_writer.hpp:457). Rank 0, first frame. Builds the triangle mesh with `BuildFaultTriangleMesh` (BuildFaultTriangleMesh, the code that `FaultHDFState::Init` had inline), opens the file and sets the filter exactly as `ParaViewHDFDataCollection` maps the deck's setting (deflate level, or ZFP accuracy). It sorts the fields by name, since the data collection writes them in that order.
- `FaultHDFStream::Save_` (fault_vtkhdf_writer.hpp:487). Rank 0, every frame. Writes the mesh entry (an unchanged mesh adds offsets only), the point data and the step.
- `VTKHDF::SetChunkToFirstAppend(bool)` and `VTKHDF::SavePointData(name, values, vdim)` in the MFEM fork (VTKHDF::SetChunkToFirstAppend, VTKHDF::SavePointData). Both are additive: nothing else in MFEM calls them, and the default behaviour of `VTKHDF` is unchanged.

```
# before: FaultVTKHDFOutput::Write -> vtkhdf::WriteFaultPackHdf, every frame
g = GatherFaultPackToRoot(pack)   # Allgather of counts, hash Bcast,
                                  # Gatherv of the vertices and of each field
rank 0: copy g into 12 L2 p=0 GridFunctions
        dc.Save(): SaveMesh; for each field: SaveGridFunction
                   (GetVectorValues per cell; append = read + inflate + deflate
                    the last 0.5 MB chunk); UpdateSteps; Flush

# after: FaultVTKHDFOutput::Write -> vtkhdf::FaultHDFStream::Write
frame 0: g = GatherFaultPackToRoot(pack); keep the cell count of every rank
         rank 0: Open_(g)  (mesh, file, filter, SetChunkToFirstAppend(true),
                            field order)
later:   Gatherv of the 10 changing fields only
rank 0:  SaveMesh (offsets only); for each field in name order
         (constant fields only at frame 0):
             SavePointData(name, cell value at its 3 points)
         UpdateSteps(t); Flush
```

What the file looks like now. The fields, steps, times, points and cells are the same, and every value read through the step offsets is bitwise equal to the old writer's (§4). Two things differ on disk: the datasets are chunked in 4704 rows instead of 65536 at the 2000 m size, and `param_a` and `param_Dc` hold one frame instead of one per step. The file size changes by −18 to +8% across the cases of §4: the constant fields stored once save space, and the smaller chunks compress a little less well.

3.2 Item 5: free what the explicit step does not use

Why. The Nitsche operator built both `ElasticityDomainOperator` objects (for K and K_η) with `SolverType::CG_AMG`. Assembly then kept the serial `ParBilinearForm` and its local matrix next to the `HypreParMatrix`, and created a BoomerAMG preconditioner and a CG solver that the explicit step never calls (elasticity_operator_assembly.inl:422 at `53ba400`). hypre runs the AMG setup only on a first solve, which never comes, so the AMG itself cost little time or memory; the serial forms are most of the saving.

What changed. A new `SolverType::OperatorOnly` (SolverType::OperatorOnly) assembles K as before and stops there: it frees the serial form, builds no solver and no AMG (elasticity_operator_assembly.inl:315). `Solve` now stops with a clear message if it is called on such an object (elasticity_operator_traction.inl:467). The Nitsche operator uses this type for K and K_η (nitsche_wave_operator.hpp:204, nitsche_wave_operator.hpp:676). Once the fused product (item 4) has copied both matrices, `ReleaseStiffness` (ReleaseStiffness) frees them too and `malloc_trim(0)` hands the memory back to the system. A diagnostic that still calls `ApplyStiffness` afterwards, such as the energy monitor, re-assembles that matrix once on its first call; no deck in the repository turns the energy monitor on.

```
# before (CG_AMG): AssembleStiffness
a.Assemble(); a.Finalize(); K = a.ParallelAssemble()    # keeps a (serial form,
                                                        #   SparseMatrix)
amg = HypreBoomerAMG(K), elasticity options; cg = CGSolver(amg)  # never used

# after (OperatorOnly): AssembleStiffness
a.Assemble(); a.Finalize(); K = a.ParallelAssemble(); free a  # no solver, no AMG
# later, UseFused_ (item 4): copy K and K_eta into the union pattern, then
ReleaseStiffness() on both; malloc_trim(0)
```

Where the peak comes from. Figure 6 in §5 shows the memory of a run over time. At p = 1 the peak is reached while K_η is assembled: K's hypre matrix, K_η's serial form and its hypre matrix exist together. Freeing the serial forms lowers it by 19% for TPV205, and the fused copy built afterwards stays under it (the TPV205 p = 1 peak is 2.46 GiB with the fused copy switched off as well). At p = 2 the peak comes later, while `FusedStiffnessPair::Init` fills the fused copy (about 20 bytes per entry) and both hypre matrices (about 12 bytes per entry each) still exist. Runs with `--fused-spmv 0` (the same binary, to 0.05 s) show it: their peak is 7.33 GiB for TPV205 and 7.97 GiB for TPV104, against 8.25 and 10.03 GiB with the fused copy and 8.90 and 9.69 GiB before. So at p = 2 the new peak, compared with the old one, is 7% lower for TPV205 and 3.5% higher for TPV104 and TPV102 on their 3000 m mesh, while the memory for stepping is 3.1 times lower. Releasing each hypre matrix as soon as its values are copied would need the two value arrays stored apart, which changes the layout the fused sweep reads; this was not done. For a run that is limited by its peak, `--fused-spmv 0` lowers the p = 2 peak by 18% against the old code and gives up item 4.

3.3 Item 3: fault traction and slip load from per-face operators

Why it was slow. Each step calls the traction twice ($T(u, d^*)$ and the Kelvin–Voigt $T_\eta(v^*, 0)$) and the slip load twice (Gd and $G_\eta V$). The old calls rebuild everything on every call and for every fault face: the face transformations, the slip at the quadrature points, the traction there, and the n_l-weighted projection onto the fault DOFs. The traction call also computes stress, correction and jump diagnostics that the step throws away, copies the whole displacement vector into a `ParGridFunction` and exchanges the face-neighbour data of the whole partition boundary.

What changed. All of that is linear in the displacement on the two elements and the slip of the face, and depends only on the fixed mesh and material. So each face gets two small dense matrices, built once by applying the production kernels to unit vectors (Figure 3).

- A_f has $3n_{bf}$ rows (dip, strike and normal traction at the face's n_{bf} fault DOFs) and $n_1 + n_2 + 2n_{bf}$ columns (displacement on element 1, on element 2, and the slip). At p = 1 it is 9 × 30, at p = 2 it is 18 × 72.
- G_f maps the face's slip to its load on the element DOFs, as `AssembleSlipFaceRHS` computes it. A shared face has only element 1's rows.

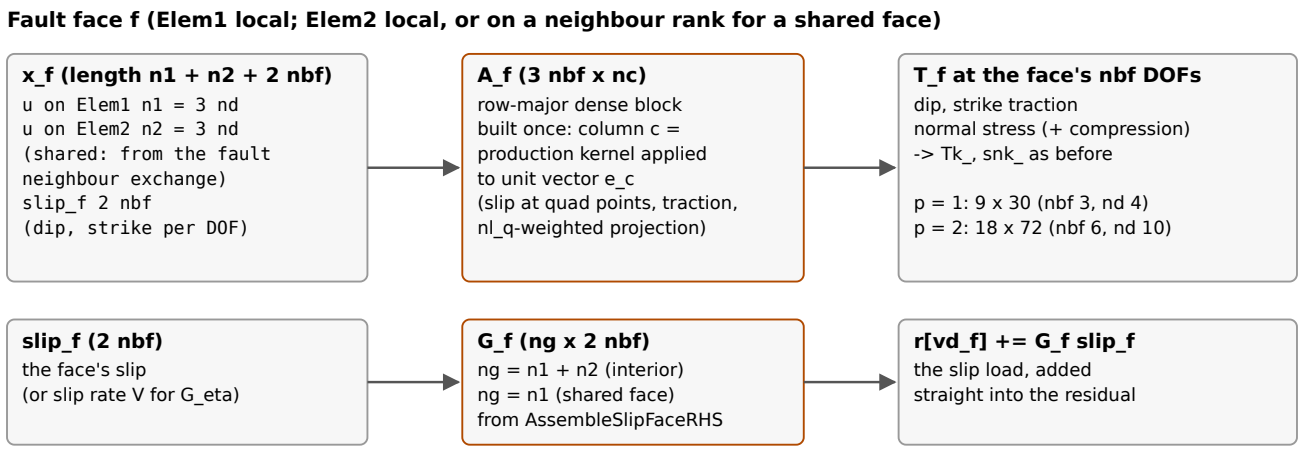


Figure 3. The two per-face operators. n_d is the number of nodes of a tet (4 at p = 1, 10 at p = 2), n_{bf} the fault DOFs of a face (3 and 6).

Functions.

- `BuildFaultFaceOps()` (BuildFaultFaceOps). Collective, called once on the first step by `UseFastFault_` (UseFastFault_), for K and for K_η. For each local fault face (interior faces first, then shared), it applies `FaultFaceTractionKernel_` (FaultFaceTractionKernel_, the IP branch of `ComputeTractionImpl`) to each unit input column to fill A_f, and the slip-load kernel to fill G_f. Output: the operators and the DOF lists, stored in `ffo_`. It took 0.17 s per operator on the 2000 m mesh.
- `BuildFaultNbrExchange_()` (BuildFaultNbrExchange_) and `FaultNbrExchange_(u)` (FaultNbrExchange_). A shared fault face needs the displacement on the neighbour rank's element. Instead of the face-neighbour exchange of every shared face, only the elements next to shared fault faces travel, between the ranks that own them.
- `ComputeTractionFast(u, slip, T, sigma_n)` (ComputeTractionFast). Input: the true-DOF displacement (or velocity for K_η) and the slip in the kernel's layout. Output: the shear and normal traction at the local fault DOFs, in the same layout as `ComputeTractionDiagnostics`.
- `AddSlipLoadFast(slip, r)` (AddSlipLoadFast). Adds $G_f\,\mathrm{slip}_f$ to the residual at the face's DOFs, straight into `F_` in `Acceleration_` (Acceleration_), with no temporary vector.

```
# setup (first step), per operator K or K_eta
for each local fault face f:
    vd_f = DOFs of element 1 (and element 2 if local)
    for each column c of x_f = [u_1, u_2, slip_f]:
        A_f[:, c] = FaultFaceTractionKernel_(e_c)
    G_f = slip-load vectors of AssembleSlipFaceRHS for unit slips
BuildFaultNbrExchange_()   # element-2 DOFs of shared fault faces, per rank

# each step: TrialTraction, KVTrialTraction (before: copy u into a ParGridFunction,
#   exchange all face-neighbour data, ComputeTractionDiagnostics on every fault face)
FaultNbrExchange_(u)
for each fault face f:  x_f = [u[vd_1], u[vd_2] or received, slip_f]
                        [T_dip; T_strike; sigma_n]_f = A_f x_f
# each step: SlipLoad, KVSlipLoad
#   (before: AssembleSlipOnlyRHS into a new vector, then F += Gd)
for each fault face f:  F[vd_f] += G_f slip_f
```

One known difference. The old slip load skips a face whose slip (or slip rate, for G_η) is below 10^{-15} at every DOF (elasticity_operator_assembly.inl:478). The matrix form adds that face's contribution, which is round-off sized. This is why the results agree to round-off rather than bitwise.

3.4 Item 4: Ku and $K_\eta v$ in one sweep

Why. The two sparse products are 84 to 91% of a step and limited by memory traffic (profiling §4.2). They read two copies of the same column indices, and each sends its own halo message.

What changed. `FusedStiffnessPair` (FusedStiffnessPair) stores the two matrices as one: each row keeps its column indices once and the pair $(K_{ij}, K_{\eta,ij})$ side by side (Figure 4). The production assembly drops explicit zeros (`Finalize(skip_zeros = 1)`), so the two patterns differ slightly; the union pattern stores a 0 where one matrix has no entry. That adds 1.8% of the entries at p = 1 and 0.1% at p = 2. The input vectors are interleaved so that one gather fetches u_j and v_j together, and the row sum uses two accumulators, because one add chain is limited by the latency of the fused multiply-add rather than by memory.

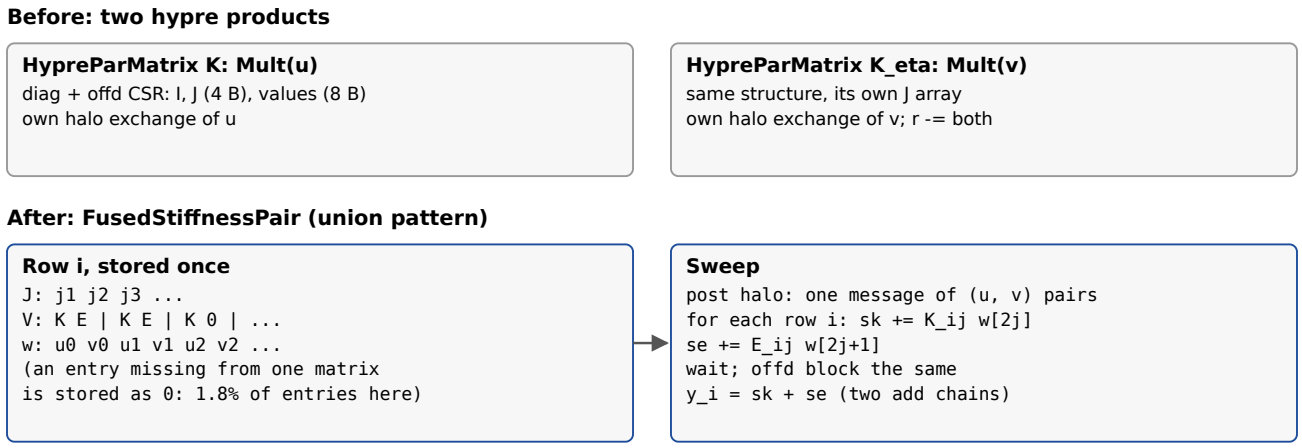


Figure 4. The two hypre products before, the fused layout and sweep after.

Functions.

- `Init(K, K_eta)` (fused_stiffness_pair.hpp:50). Input: the two assembled `HypreParMatrix` objects with the same rows. It maps one off-diagonal column map into the other (`SubsetMap_`, fused_stiffness_pair.hpp:167; the larger one hosts), builds the union of the diagonal and off-diagonal blocks in two passes (`Union_`, fused_stiffness_pair.hpp:190: count, then fill, so no array grows by doubling), and copies the host matrix's communication package. It returns false if the row partitions or column maps do not fit, and the operator then keeps the two products.
- `Mult(u, v, y)` (fused_stiffness_pair.hpp:102). Output: $y = Ku + K_\eta v$. It posts the halo exchange of the (u, v) pairs, sweeps the diagonal block while the messages travel, waits, then sweeps the off-diagonal block.
- `UseFused_()` (UseFused_). Collective on the first step: `Init` on every rank, an `MPI_Allreduce` of the outcome so all ranks take the same path, then the release of item 5.

```
# before: Acceleration_
K.Mult(x, Kx)          # hypre: halo of x, diag + offd sweep
F -= Kx
...
K_eta.Mult(v, Kv)      # hypre: halo of v, diag + offd sweep
F -= Kv

# after: Acceleration_ -> FusedStiffnessPair::Mult(x, v, Kx)
post Irecv; pack (x_j, v_j) for the send rows; Isend   # one message per neighbour
w[2j] = x_j; w[2j+1] = v_j                             # interleave
for each row i:  sk = se = 0
    for p in row i of the diag block:
        J = col[p];  sk += V[2p] w[2J];  se += V[2p+1] w[2J+1]
    Kx_i = sk + se
Waitall; the offd block the same way with the received pairs
F -= Kx                # Kx now holds K x + K_eta v
```

3.5 Item 6: fault-weighted partition

Why. A rank that owns fault faces does the fault work on top of its share of tets. METIS balances tets only, so the fault ranks set the pace (profiling §4.4).

What changed. With `--fault-weight w` above 0, the driver gives each tet the METIS weight $100(1 + w\,n_f)$, where n_f is its number of fault faces, and cuts with the repository's `BuildLtsAwarePartition` (one cluster, imbalance tolerance 1.03, which is METIS's k-way default and so matches MFEM's own cut) (driver partition). With $w = 0$ the driver calls `ParMesh` exactly as before. A run prints its own estimate of w (cost model):

$$c_{\text{tet}} = \frac{\sum_r (\text{step time}_r - \text{fault time}_r)}{(N-1)\,N_{\text{tet}}}, \qquad c_{\text{side}} = \frac{\sum_r \text{fault time}_r}{(N-1)\sum_r (2n_r^{\text{int}} + n_r^{\text{sh}})}, \qquad w = \frac{c_{\text{side}}}{c_{\text{tet}}},$$

where the times are summed over steps 2 to N (step 1 also assembles the operators), the fault time is `AdvanceFault_` up to its reductions plus the two slip loads, and an interior fault face has two sides on its rank and a shared face one. On more than one rank the fault time also holds waits for neighbours, so a 1-rank run gives the cleanest w (§6).

4. Checks: the results are unchanged

Every case of §5 was compared with the `53ba400` binary on the same mesh, deck and rank count. Those runs are short, and TPV102 and TPV104 at p = 2 end while the fault is still nucleating, so TPV205 and TPV104 at p = 1 were also run to 3 s, and TPV205 at p = 1 on one rank. In the 3 s runs the TPV104 rupture grows past its nucleation zone (42 of 672 fault output points pass 1 mm/s, the last at 2.75 s), while on the coarse 2000 m mesh the TPV205 rupture stays in its nucleation patch and stops (44 of 1176 fault output points pass 1 mm/s, the last at 0.77 s). Table 3 gives the largest station difference, measured per quantity: the difference over the run divided by the largest value of that quantity at that station (slip, slip rate and shear traction take both components together). A per-column measure is larger only in columns that are zero in exact arithmetic: the dip components of these strike-slip runs are 10^{-31} m of slip and 10^{-6} Pa of traction against 40 MPa, and differ by 10^{-35} m and 10^{-10} Pa.

Case	Peak slip rate (m/s)	Stations	Largest station difference, per quantity	Largest fault-file difference	Same steps, points, cells
TPV205, p = 1, 2000 m, 1.0 s, 4 ranks	0.19	16	$6.2 \cdot 10^{-14}$ (V)	$1.7 \cdot 10^{-13}$	yes
TPV205, p = 1, 2000 m, 1.0 s, 1 rank	0.19	16	$1.4 \cdot 10^{-15}$ (tau)	$1.3 \cdot 10^{-13}$	yes
TPV205, p = 2, 3000 m, 0.5 s, 4 ranks	0.17	16	$1.2 \cdot 10^{-14}$ (tau)	$6.8 \cdot 10^{-14}$	yes
TPV104, p = 1, 3000 m, 1.0 s, 4 ranks	0.49	9	$1.4 \cdot 10^{-11}$ (slip)	$8.6 \cdot 10^{-15}$	yes
TPV104, p = 2, 3000 m, 0.5 s, 4 ranks	$6.0 \cdot 10^{-3}$	9	$2.0 \cdot 10^{-11}$ (slip)	$1.3 \cdot 10^{-10}$	yes
TPV102, p = 1, 1000 m, 0.5 s, 4 ranks	$1.6 \cdot 10^{-4}$	9	$1.0 \cdot 10^{-25}$ (V)	$1.0 \cdot 10^{-15}$	yes
TPV102, p = 2, 3000 m, 0.5 s, 4 ranks	$3.6 \cdot 10^{-3}$	9	$1.3 \cdot 10^{-26}$ (tau)	$4.6 \cdot 10^{-16}$	yes
TPV205, p = 1, 2000 m, 3.0 s, 4 ranks	0.19	16	$6.2 \cdot 10^{-14}$ (V)	$1.7 \cdot 10^{-13}$	yes
TPV104, p = 1, 3000 m, 3.0 s, 4 ranks	2.9	9	$9.7 \cdot 10^{-11}$ (V)	$3.0 \cdot 10^{-14}$	yes

Table 3. Old against new, relative differences. The quantity named is where the largest station difference sits. The peak slip rate shows how far the rupture got: below about 0.01 m/s the run ended during nucleation.

The fault file was compared step by step through the step offsets, as ParaView reads it (`cmp_vtkhdf.py`). Times, fields, points and cell types are identical in every case, and the values agree to $1.3 \cdot 10^{-10}$ at most. The new files differ in size from the old ones by −18 to +8% (TPV205 p = 1, 2000 m: 3.44 MB against 3.53 MB). With the fast output switched off (`--fast-fault-output 0`), the stations and every fault-file value are bitwise identical to the run with it on, so item 1 changes nothing but the file layout. The weighted partition reorders the cells (the gather follows rank order), so that file was compared after matching cells by their corner coordinates (`cmp_vtkhdf_sorted.py`): all 1568 cells matched, values within $1.5 \cdot 10^{-13}$, and the stations within $6.2 \cdot 10^{-14}$.

The bench (`seas_bench_nitsche_kernels`) checks each new call against the call it replaces on the production operators. All 13 checks pass on 1 rank and all 7 on 4 ranks (2000 m TPV205

mesh, p = 1), and all 11 at p = 2 on 1 rank (5000 m TPV205 mesh): `FusedStiffnessPair::Mult` equals the two `ApplyStiffness` calls to 0, `ComputeTractionFast` equals `ComputeTractionDiagnostics` to $5.5 \cdot 10^{-16}$ and `AddSlipLoadFast` equals `AssembleSlipOnlyRHS` to $2.8 \cdot 10^{-16}$.

5. Measured gains

Setup of the runs. Each case ran on 4 ranks with `OMP_NUM_THREADS=1` and the deck's fault output every 0.02 s of model time. The decks are the repository's `tpv205/configs/tpv205_nitsche.toml`, `tpv104/configs/tpv104_nitsche.toml` and `tpv102/configs/tpv102_nitsche_pc_p1_kv0p1.toml` (Kelvin–Voigt damping 0.1, Prakash–Clifton on), with one line changed: `snap_tolerance_m` from 700 to 3000 m, because the coarse meshes have no fault node within 700 m of some stations. `--order` sets p and `--tfinal` the end time. "Setup" is the wall time extrapolated back to step 0 from the steady step rate, so it includes the assembly that happens inside the first step. "Per step" is the steady wall time per step without fault output. Memory is the summed resident memory of the 4 ranks, sampled every 0.2 s. The VM's run-to-run noise is about 10%, so the two TPV205 cases ran 3 and 2 times, alternating old and new, and Table 4 gives the median; the other cases ran once.

Case (4 ranks)	Code	Wall (s)	Setup (s)	Per step, no output (ms)	Output per frame (s)	Peak memory (GiB)	Memory while stepping (GiB)	Speedup
TPV205, p = 1, 2000 m, 1.0 s	53ba400	39.0	6.4	69.9	0.1289	3.03	3.03	
	new	27.5	6.3	56.2	0.0063	2.45	1.18	1.42×
TPV205, p = 2, 3000 m, 0.5 s	53ba400	51.0	19.1	137.0	0.1144	8.90	8.90	
	new	43.7	19.2	115.8	0.0038	8.25	2.84	1.17×
TPV104, p = 1, 3000 m, 1.0 s	53ba400	17.7	3.0	35.2	0.1309	1.63	1.63	
	new	9.8	3.3	28.3	0.0060	1.39	0.67	1.82×
TPV104, p = 2, 3000 m, 0.5 s	53ba400	63.3	25.4	150.3	0.1214	9.69	9.69	
	new	57.6	25.4	138.2	0.0057	10.03	3.09	1.10×
TPV102, p = 1, 1000 m, 0.5 s	53ba400	143.8	38.0	300.7	0.1906	10.14	10.14	
	new	119.9	31.0	262.4	0.0278	8.68	3.76	1.20×
TPV102, p = 2, 3000 m, 0.5 s	53ba400	68.6	24.5	174.3	0.1482	9.69	9.69	
	new	56.3	26.7	127.2	0.0060	10.03	3.09	1.22×
TPV205, p = 1, 2000 m, 3.0 s	53ba400	101.0	5.9	68.0	0.1264	3.03	3.03	
	new	66.4	5.8	53.9	0.0064	2.60	1.18	1.52×
TPV104, p = 1, 3000 m, 3.0 s	53ba400	46.9	2.7	33.7	0.1377	1.63	1.63	
	new	20.5	2.6	25.8	0.0063	1.33	0.67	2.29×

Table 4. Old and new on each case. TPV205 rows are medians of 3 (p = 1) and 2 (p = 2) alternating runs; the other rows are single runs. The last two rows ran to 3 s.

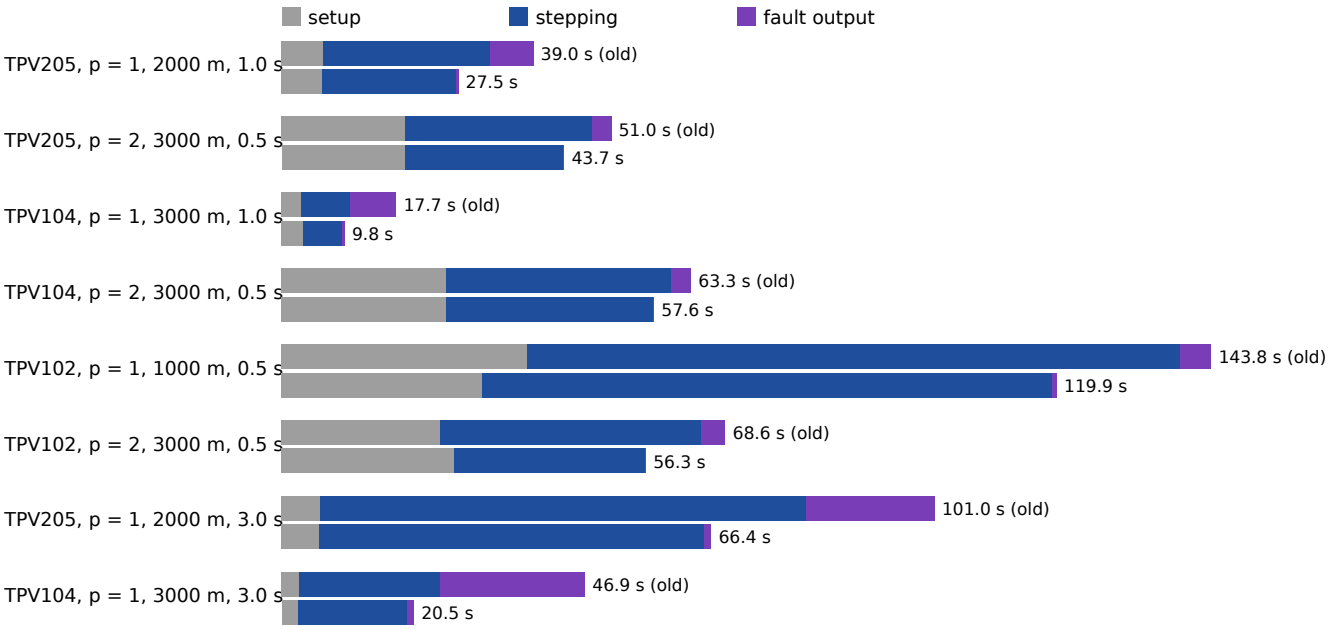


Figure 5. Wall time of each case, old (upper bar) and new (lower bar), split into setup, stepping and fault output.

Each item alone. The TPV205 cases were also run with one item switched off at a time (Table 5). The difference to the full new run is that item's share. The shares do not add up exactly to the total, because of the VM's noise and because the items overlap: item 3 removes the whole-boundary face-neighbour exchange, which also shortens the time other ranks wait in the fused halo exchange.

Case	Run	Per step, no output (ms)	Change, share of the old step	Output per frame (s)	Wall (s)	Memory while stepping (GiB)
TPV205, p = 1	All five on	56.8		0.0067	28.0	1.18
	Item 1 off (--fast-fault-output 0)	57.6	+1%	0.1304	34.2	1.18
	Item 3 off (--fast-fault 0)	69.6	+17%	0.0070	33.1	1.19
	Item 4 off (--fused-spmv 0)	65.1	+11%	0.0067	30.6	2.46
	Item 6 on (--fault-weight w = 1.33)	57.8	+1%	0.0068	28.3	1.18
TPV205, p = 2	All five on	119.4		0.0042	48.7	2.84
	Item 1 off (--fast-fault-output 0)	119.5	+0%	0.1165	53.8	2.84
	Item 3 off (--fast-fault 0)	138.7	+13%	0.0046	50.3	2.87
	Item 4 off (--fused-spmv 0)	132.6	+9%	0.0040	48.2	7.33
	Item 6 on (--fault-weight w = 1.01)	115.2	-3%	0.0041	50.3	2.84

Table 5. One item switched off at a time (single runs, 4 ranks). The change is the extra time per step over the full new run, as a share of the old step. With item 4 off the hypre matrices stay and `malloc_trim` is not called, so the stepping memory stays high. Item 6 used the w that a 4-rank run of the new binary printed, which still counted neighbour waits as fault time (§3.5); the 1-rank values of §6 are smaller.

Memory over time. In the old TPV205 p = 2 run the memory climbs to 8.90 GiB during setup and stays there, because the serial forms and the matrices are all kept. The new run reaches 8.25 GiB while the fused copy is filled next to the two hypre matrices, then drops to 2.84 GiB when the fused copy replaces them and `malloc_trim` returns the freed memory (Figure 6). The dashed run has the fused copy switched off: its setup peak is 7.33 GiB, but it keeps both hypre matrices while stepping.

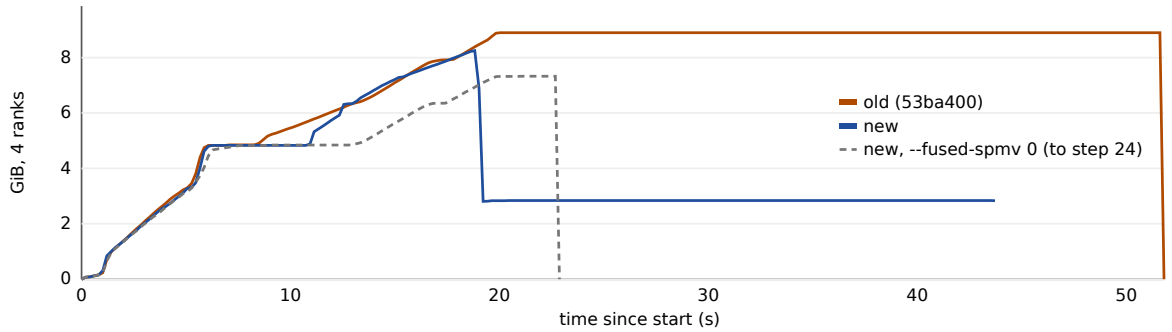


Figure 6. Summed resident memory of the 4 ranks during the TPV205 p = 2 run, old and new, and the new binary with the fused copy switched off (run to 0.05 s, so its curve ends after setup).

One rank. On one rank, TPV205 p = 1 (2000 m, 1 s) takes 119.9 s before and 102.2 s after (1.17 times faster). A step without output takes 250 ms before and 217 ms after, 13% less, against 20% on 4 ranks: on 4 ranks item 3 also removes waits in the face-neighbour exchange.

Fault output at the 200 m size. The bench writes 20 frames of 12 fields on 104 882 triangles, about the size of the 200 m TPV205 fault, on one rank with deflate 1 (Table 6). The SAFS writer takes 1.15 s per frame and the new writer 0.454 s, 2.5 times less. Vista measured 0.48 s per frame for the old writer at 200 m (profiling §4.1), so this VM's rank 0 is about 2.4 times slower. If that ratio carries over, the new writer would take about 0.19 s per frame on Vista, some 113 s for 600 frames instead of 290 s (inferred). At this size most of what is left is deflate itself: with no filter the new writer takes 0.023 s per frame, 19.7 times less again, and the bench file is 2.6 times larger. On the real files of the 3 s runs of Table 4, deflate 1 stores the fault fields in 1/2.7 (TPV104) and 1/5.9 (TPV205, whose rupture stops early) of their raw size, so with no filter they would be that much larger. The decks cannot choose that today: a negative `paraview_fault_deflate_level` falls back to level 6 (nitsche_dyn_driver.cpp). Adding a "no filter" setting is a small change that this work does not make: it trades output time for disk, so it is left to Chunhui Zhao.

Writer	1568 triangles (2000 m) (ms per frame)	File (MB)	104 882 triangles (about 200 m) (ms per frame)	File (MB)
ParaViewHDFDataCollection::Save, deflate 1 (the decks)	124.6		1051.0	
the same, deflate 0	87.5		669.2	
the same, no filter	15.2		558.9	
SAFS writer (old driver path), deflate 1	124.1	2.1	1153.0	236.4
SAFS writer, no filter	13.8	32.1	589.7	631.9
FaultHDFStream (new), deflate 1	8.5	6.1	453.5	199.2
FaultHDFStream, no filter	1.0	15.5	23.1	524.9

Table 6. Fault frame cost on one rank, bench [H], 12 fields: the mean of the frames after the first (40 and 20 frames written), and the size of the file at the end. The first three rows are the data collection alone, as the profiling report measured it; the last four are the production calls. The bench fields are sine waves, so their file sizes are only a rough guide; the text above gives the deflate ratio of the real fault files, and §4 their sizes.

6. Partition weight on the 200 m meshes

The cost model of §3.5 from 1-rank runs of the 2000 m and 3000 m meshes gives c_{tet} = 3.47 µs and c_{side} = 1.29 µs at p = 1, so w = 0.37, and 14.7 µs and 3.7 µs at p = 2, so w = 0.25. After item 3 a fault-face side costs about as much as 0.3 to 0.4 tets; before it, the profiling report measured about 6 tets at p = 1 and 2.5 at p = 2 (profiling §4.4, 41.5 and 77.6 µs per face, two sides). The bench's [P] section cuts the 200 m meshes with MFEM's default and with the weighted cut, counts tets and fault-face sides per part, and predicts the step time of the slowest part over the mean (Table 7). It assumes the per-tet and per-side costs above and leaves out communication.

Mesh	Tets, fault faces	Ranks (as the Vista golds)	Ranks with fault faces	Most fault-face sides on a rank	Predicted max/mean, MFEM cut	Predicted max/mean, weighted	Predicted gain
TPV205 200 m, p = 1	1.90 M, 26642	576	28%	650	1.08	1.03	1.05×
TPV205 200 m, p = 2	1.90 M, 26642	2304	20%	231	1.09	1.03	1.06×
TPV104 200 m, p = 1	2.46 M, 37920	512	34%	820	1.06	1.03	1.03×
TPV104 200 m, p = 2	2.46 M, 37920	2048	24%	310	1.07	1.03	1.04×

Table 7. Bench [P] on the 200 m meshes with the costs of the 1-rank runs above, MFEM's default cut against the weighted cut with that w. A max/mean of 1 is perfect balance.

Under MFEM's cut the slowest rank is a fault rank. Its tets alone are balanced to about 1.02 of the mean, and its fault-face sides add the rest. The weighted cut brings all four cases to 1.03. That is the tolerance the driver passes to METIS (`popt.imbalance = 1.03`, METIS's k-way default), so METIS stops there and the gain is capped at 1.03 to 1.06×. The Vista run of §7 measured it on TPV205 200 m p = 2 with `--fault-weight 0.25`: the weighted max/mean came out at 1.030, the step went from 16.38 to 16.26 ms (0.7% faster), the results stayed the same to round-off, and the first node's peak memory rose from 146 to 161 GB.

7. Vista: TPV205 200 m, p = 2, on 2304 ranks

What ran. Build job 1078824 built commit `6d41ba7` in a fresh clone (`$WORK/seas-mfem-profiling`), and all six Nitsche gate test sets passed there (74, 22, 49, 13, 9 and 42 checks, none failed). Speed job 1078825 then ran the deck and mesh of gold 1075916 (`tpv205/configs/tpv205_nitsche.toml`, `tpv2053d_200m.msh` with its sha256 checked) to 1.0 s on the gold's 16 nodes × 144 ranks, three times in one allocation: with `--fast-fault 0 --fused-spmv 0 --fast-fault-output 0` (the old calls; item 5 stays on), with the defaults, and with the

defaults plus `--fault-weight 0.25`. The job took 14 min, about 1.2 SU of EAR26024. The job scripts are in `miniapps/seas/jobs/vista_profiling/`.

Run	Step, slowest rank (ms)	Stepping per rank, min / mean / max (s)	Fault work, busiest rank (ms per step)	Fault frame (s)	Rest of the wall time (s)	Wall (s)	First node peak (GB)	Stations
old calls (flags off)	27.00	149 / 186 / 200	7.67	0.406	101	321	147	bitwise equal to gold 1075916 (16 of 16)
new calls (defaults)	16.38	89 / 110 / 121	3.30	0.127	115	244	146	$6.6 \cdot 10^{-11}$ against the old calls
new calls, weighted cut (w = 0.25)	16.26	87 / 111 / 120	3.66	0.124	114	242	161	$6.6 \cdot 10^{-11}$ against the old calls

Table 8. Vista job 1078825: TPV205 200 m, p = 2, 0 to 1.0 s (7394 steps, 49 fault frames), 16 nodes × 144 ranks, commit `6d41ba7`. "Step" and "stepping" are the driver's `ode.Step` timers after step 1. "Rest" is the wall time minus the stepping of the slowest rank (with step 1) and the fault output, so it is mostly setup. Memory is the first node's (which holds rank 0 and the fault output), sampled every 2 s out of 238 GB. "Stations" is the largest relative station difference per quantity.

The results are unchanged. With the old calls the run reproduces the first second of gold 1075916 exactly: all 16 stations, 7394 rows each, are bitwise equal as printed. So with the flags off the new build follows the gold's code path exactly. With the new calls, and with the weighted cut, the largest station difference against the old calls is $6.6 \cdot 10^{-11}$ (slip at x2_0_x3_7.5), round-off like the cloud checks of §4. Peak slip rate (3.7397 m/s), the normal-stress range and the step count are the same in all three runs.

Where the time went. The step of the slowest rank falls from 27.0 to 16.4 ms (1.65×), and the mean rank's stepping from 186 to 110 s (1.68×). The driver's fault-work timer on the busiest rank falls from 7.67 to 3.30 ms per step; that drop is item 3, and it is 4.4 of the 10.6 ms saved. The rest comes from the fused products (item 4) and from shorter waits for the fault ranks, but Vista did not run the items one at a time, so the split between those two is not measured. A fault frame falls from 0.406 to 0.127 s (3.2×), of which the gather is 3.8 ms and the HDF5 write 0.123 s. The rest of the wall time, mostly setup, is 101 s with the old calls and 115 s with the new ones: setup does not get faster, and is about 14 s longer, which fits the new calls building the per-face operators and the fused copy there (not timed separately on Vista).

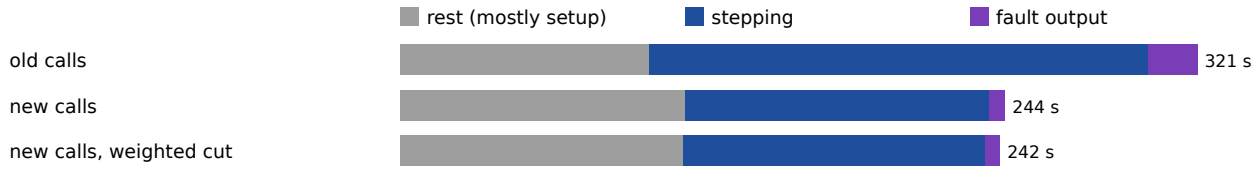


Figure 7. Wall time of the three Vista runs (1.0 s of model time), split into the rest (mostly setup), the stepping of the slowest rank and the fault output.

A full-length run. Gold 1075916 ran the old calls to 12 s: 88 725 steps and 599 frames in 2670 s. With the new calls, the 1 s run's costs (115 s of setup, 16.38 ms per step, 0.127 s per frame) give about 1640 s for the same run, 1.62 times faster, which saves about 1.5 of the gold's 3.9 SU. This is an estimate: it assumes the step and frame costs stay as in the first second. The gold's own step rate did not grow later in the run, but its frames did cost more (0.49 s against 0.41 s in the first second).

Partition weight. The weighted cut reached a weighted max/mean of 1.030, the METIS tolerance of §6, and the step went from 16.38 to 16.26 ms, 0.7% faster. Its busiest rank does more fault work (3.66 against 3.30 ms), and the first node's peak memory is 15 GB higher (161 against 146 GB). That gain is smaller than the 1.03 to 1.06× that §6 predicted, so the default stays at 0.

The bench on 16 ranks. In build job 1078824 the bench crashed with a segmentation fault on 4 of the 16 ranks, right after its [V] section. [V] times MFEM partial assembly, which calls `Mesh::EnsureNodes`; on a ParMesh without nodes that runs `ParMesh::SetCurvature`, whose first line deletes the mesh's face-neighbour data (`mesh/pmesh.cpp:2019`). The traction calls of [T] that came next need that data on the ranks with shared fault faces, and crashed in `ParMesh::GetFaceNbrElementTransformation`. It is a bench-only fault: the driver never calls partial assembly, and the cloud multi-rank runs of §4 had used `--skip-volume`. Commit `0c83fb5` moves [V] after [T2], and the rerun (job 1078847) passes all checks (Table 9), including the per-face operators of item 3 across shared fault faces.

Bench section and call	Time per call, 16 ranks	Relative difference
[K] Ku, production `ApplyStiffness`	10.67 ms (45% of the 21.7 GB/s triad per rank)	reference
[F2] `FusedStiffnessPair::Mult` against the two `ApplyStiffness` calls	14.21 ms against 14.54 ms (min); 14.61 against 17.92 ms (median): 1.024× and 1.227×	equal as printed
[T2] `ComputeTractionFast` against `ComputeTractionDiagnostics`	0.04173 ms against 1.607 ms (38.51×)	$5.2{\cdot}10^{-16}$ shear, $6.8{\cdot}10^{-16}$ normal
[T2] `AddSlipLoadFast` against `AssembleSlipOnlyRHS`	0.01817 ms against 0.8537 ms (46.98×)	$3.3{\cdot}10^{-16}$
[V] volume term: partial assembly, dense blocks against CSR	1.084× and 1.021×	equal as printed

Table 9. Bench on 16 Vista ranks (job 1078847, commit `0c83fb5`), TPV205 1000 m mesh made on Vista, p = 1, 20 repetitions, 10 shared fault faces. All 9 checks pass.

Fault writer at the 200 m size. Bench [H] on one Vista rank (job 1078847, the same frames as Table 6) gives 546 ms per frame for the old writer and 231 ms for the new one with deflate 1, and 247 and 6.6 ms with no filter (files of 236 and 199 MB with deflate, 632 and 525 MB without). In the production run the new writer's frame took 0.127 s. Fault output is now 5% of the stepping time, so writing without deflate would save at most a few percent of a run and make the fault file about 2.6 times larger; the deck setting stays as it is.

8. Limits and next steps

- **Machine.** §4 to §6 are from the cloud VM, whose product Ku reaches 48 to 56% of the triad bandwidth (bench [S] and [K]). Vista measured only TPV205 200 m at p = 2 (§7). TPV104, TPV102 and p = 1 have not been run on Vista with the new calls; they use the same kernels, but their gains there are not measured.
- **Round-off.** The results are not bitwise equal to `53ba400`: the products sum in a different order, the slip load no longer skips faces below 10^{-15} m, and the per-face traction sums in a different order. The golds stay valid as references at their stated tolerances; a new gold is not needed.
- **Fault file layout.** Constant fields are stored once and chunks are frame-sized. ParaView and the repository's checker read it unchanged. A script that reads `PointData/param_a` directly, without the step offsets, sees one frame instead of one per step.
- **Peak memory.** Only the stepping memory fell a lot. The setup peak fell 14 to 19% at p = 1; at p = 2 it is 7% lower for TPV205 and 3.5% higher for TPV104 and TPV102 on their 3000 m mesh, because the fused copy is built next to both hypre matrices (§3.2). On Vista the first node's peak was 147 GB with the old calls and 146 GB with the new ones (2 s samples, which can miss a short peak).
- **Energy monitor.** If `[diagnostics] energy_balance` is on, the monitor re-assembles K and K_η on its first call and keeps them, which gives back the memory saving but not the speed.
- **Items 7 and 8** wait on Chunhui Zhao's decision after the design document (`lts-kernels-design-rev1.pdf`). Item 8a would replace the fused CSR sweep of item 4 with dense blocks, so item 4's union layout is a stepping stone rather than the end point.

9. Where it lives

- This report: `/mnt/project-files/profiling/nitsche-speedups-impl-rev1.1.pdf`, with `nitsche-speedups-impl-rev1.1-source.html` beside it. Revision 1 stays beside it.
- Code: branch `claude/nitsche-profiling-w336yq` on GitHub, on `b5427c8` (the bench) on `53ba400` (`nitsche-benchmarks`). The change is two commits: a structural one (+43 −35 lines) that factors `BuildFaultTriangleMesh` out, and the speedups (+1526 −53 lines), committed as `5cb2500` (the `BuildFaultTriangleMesh` factoring) and `2673c50` (the items), then `6d41ba7` (the Vista jobs, `miniapps/seas/jobs/vista_profiling/`) and `0c83fb5` (the bench fix of §7), all reviewed by Chunhui Zhao before the commit. Files: `mesh/vtkhdf.hpp`, `mesh/vtkhdf.cpp` (MFEM fork), and under `miniapps/seas/`: `domain/elasticity_operator.hpp`, `domain/elasticity_operator_assembly.inl`, `domain/elasticity_operator_traction.inl`, `domain/elasticity_operator_fault_ops.inl` (new), `domain/fused_stiffness_pair.hpp` (new),

`domain/nitsche_wave_operator.hpp`, `io/fault_vtkhdf_writer.hpp`, `drivers/nitsche_dyn_driver.cpp`, `tests/unit/bench_nitsche_kernels.cpp`, `Makefile`.

- Runs: the cloud VM, session scratchpad `impl/`: `matrix.py` (all cases and the one-item-off runs), `repeat.py` (the repeats and the 1-rank runs), `run_case.py` (one run with its memory samples), `cmp2.py`, `cmp_vtkhdf.py`, `cmp_vtkhdf_sorted.py`, and the logs and outputs under `impl/runs/`. The meshes are coarse versions of the repository geometries, regenerated in the scratchpad (`mesh/`); TPV102 1000 m is the repository's `tpv102/mesh/tpv102_1000m.msh`. The one-item-off runs used a build whose only difference from the reviewed tree is the timing printout of §3.5.
- Vista: `/mnt/project-files/profiling/vista-runs/` holds the stdout of jobs 1078825 and 1078847 and a README; the run outputs stay in Vista `$SCRATCH/nitsche_profiling/tpv205_200m_p2_speed_1078825/` (scratch, purged).
- Bench output: `impl/bench_np1.txt`, `impl/bench_np4.txt`, `impl/bench_np1_p2.txt` (5000 m, p = 2), `impl/bench_h.txt`, `impl/bench_h_small.txt`, `impl/bench_p_*.txt` in the scratchpad. The `--fused-spmv 0` memory runs are `impl/runs/x_*_fused0`.